End-to-End Execution Guide v5

Casting Defect Analysis Pipeline, REST API Serving & Streamlit Web UI with Advanced MLOps & Transfer Learning

PHASE 1: PROJECT SETUP & DIRECTORIES

■ Step 1: Create & Enter Project Folder

> *Description: Initializes the root workspace directory for the casting defect analysis project and navigates into it.*

```
mkdir DefectAnalyser
cd DefectAnalyser
```

■ Step 2: Create Training Folder

> *Description: Sets up an isolated workspace directory specifically designated for model training scripts and experimentation.*

```
mkdir training
```

■ Step 3: Setup Data Folder & Kaggle Dataset

> *Description: Organizes the raw manufacturing casting imagery dataset downloaded from Kaggle into structured train and test subfolders categorized by defect classes (def_front vs. ok_front).*

```
data/
└── casting_data/
    └── casting_data/
        ├── train/ (def_front / ok_front)
        └── test/ (def_front / ok_front)
```

PHASE 2: ENVIRONMENT INITIALIZATION

■ Step 4 & 5: Python 3.12 & Virtual Environment

> *Description: Verifies Python 3.12 installation and provisions an isolated virtual environment (`qualityenv`) to prevent dependency conflicts with system packages.*

```
py install 3.12
py -3.12 -m venv qualityenv
```

PHASE 3: ACTIVATION & DEPENDENCIES

■ Step 6, 7 & 8: Activation & Core Dependencies Install

> *Description: Configures PowerShell execution policy, activates the virtual environment, and installs core deep learning dependencies including TensorFlow.*

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process
# Example alternative scope if needed:
# Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
.\qualityenv\Scripts\Activate.ps1
pip install tensorflow
```

PHASE 4: CODE & EXECUTION

■ Step 9: Point VS Code Interpreter

> *Description: Configures the Visual Studio Code workspace interpreter to point to the active virtual environment (`qualityenv`) via Command Palette (`Ctrl + Shift + P`).*

```
# Select Python 3.12.x (qualityenv : venv) in VS Code via Ctrl + Shift + P
```

■ Step 10: Custom CNN Training Script (`training/training_model.py`)

> *Description: Implements a custom Convolutional Neural Network with data augmentation layers, dynamic input shape configuration, early stopping regularization, and absolute pathing.*

```
import os
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.callbacks import EarlyStopping

CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
BASE_DIR = os.path.dirname(CURRENT_DIR)
DATA_DIR = os.path.join(BASE_DIR, "data", "casting_data", "casting_data")
TRAIN_DIR = os.path.join(DATA_DIR, "train")
TEST_DIR = os.path.join(DATA_DIR, "test")
MODEL_STORE_DIR = os.path.join(BASE_DIR, "model_store")

# Hyperparameters (Easily tunable)
IMG_SIZE = (512, 512)
BATCH_SIZE = 16
EPOCHS = 5
LEARNING_RATE = 1e-4

def run_training_pipeline():
    os.makedirs(MODEL_STORE_DIR, exist_ok=True)

    # Load Datasets
    train_ds = tf.keras.utils.image_dataset_from_directory(
        TRAIN_DIR, image_size=IMG_SIZE, batch_size=BATCH_SIZE, shuffle=True
    )
    test_ds = tf.keras.utils.image_dataset_from_directory(
        TEST_DIR, image_size=IMG_SIZE, batch_size=BATCH_SIZE, shuffle=False
    )

    # Build Model with Data Augmentation & Tunable Hyperparameters
    model = models.Sequential([
        layers.Input(shape=(512, 512, 3)),

        # Data Augmentation Layers (Applied only during training)
        layers.RandomFlip("horizontal_and_vertical"),
        layers.RandomRotation(0.2),
        layers.RandomContrast(0.1),

        # Rescaling pixel values
        layers.Rescaling(1./255),

        # Convolutional Blocks
        layers.Conv2D(32, (3, 3), activation='relu'),
        layers.MaxPooling2D((2, 2)),

        layers.Conv2D(64, (3, 3), activation='relu'),
        layers.MaxPooling2D((2, 2)),

        layers.Conv2D(128, (3, 3), activation='relu'),
        layers.MaxPooling2D((2, 2)),

        layers.Flatten(),
        layers.Dense(128, activation='relu'),
        layers.Dropout(0.4),  # Tunable dropout rate
```

```
        layers.Dense(1, activation='sigmoid')
    ])

    # Compile with configurable learning rate
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=LEARNING_RATE),
        loss='binary_crossentropy',
        metrics=['accuracy']
    )

    # Early Stopping callback to halt training when validation performance plateaus
    early_stopping = EarlyStopping(
        monitor='val_loss',
        patience=5,
        restore_best_weights=True
    )

    # Train model
    model.fit(
        train_ds,
        validation_data=test_ds,
        epochs=EPOCHS,
        callbacks=[early_stopping]
    )

    # Save the updated model
    model_path = os.path.join(MODEL_STORE_DIR, "casting_defect_classifier.keras")
    model.save(model_path)
    print(f"Optimized model saved to: {model_path}")

if __name__ == "__main__":
    run_training_pipeline()
```

■ Step 11: Execute Training Script

> *Description: Kicks off the baseline custom CNN model training pipeline using the training data directory.*

```
python training/training_model.py
```

PHASE 5: MODEL DEPLOYMENT & SERVING (REST API)

■ Step 12 & 13: Serving Setup & Dependencies

> *Description: Creates the serving module directory and installs FastAPI, Uvicorn, Pillow, and NumPy for robust backend web services.*

```
mkdir serving
pip install fastapi uvicorn pillow numpy
```

■ Step 14: REST API Implementation (`serving/api.py`)

> *Description: Builds a production-grade FastAPI application featuring a health check endpoint, direct multipart image file uploads, tensor preprocessing, and defect classification inference.*

```
import os
import numpy as np
from fastapi import FastAPI, HTTPException, UploadFile, File
from pydantic import BaseModel
import tensorflow as tf
from PIL import Image
import io

# Initialize FastAPI app
app = FastAPI(
    title="Casting Defect Classification API",
    description="REST API for serving the casting defect classifier model with direct image
uploads.",
    version="1.1"
)

# Define paths relative to project structure
CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
BASE_DIR = os.path.dirname(CURRENT_DIR)
MODEL_PATH = os.path.join(BASE_DIR, "model_store", "casting_defect_classifier.keras")

# Load Model & Feature Schema
print(f"Loading model from: {MODEL_PATH}")
if os.path.exists(MODEL_PATH):
    model = tf.keras.models.load_model(MODEL_PATH)
else:
    raise FileNotFoundError(f"Model file not found at {MODEL_PATH}. Please run training
first.")

IMG_SIZE = (512, 512)

class InferenceResponse(BaseModel):
    prediction: float
    result: str

@app.get("/")
def health_check():
    return {"status": "healthy", "model_loaded": True}

@app.post("/predict", response_model=InferenceResponse)
async def predict_defect(file: UploadFile = File(...)):
    try:
        contents = await file.read()
        image = Image.open(io.BytesIO(contents)).convert("RGB")
        image = image.resize(IMG_SIZE)

        img_array = np.array(image, dtype=np.float32)
        img_array = np.expand_dims(img_array, axis=0)

        prediction = model.predict(img_array)
        score = float(prediction[0][0])
        result_label = "ok" if score >= 0.50 else "defective"

        return InferenceResponse(prediction=score, result=result_label)
```

```
    except Exception as e:
        error_msg = str(e.__dict__) if hasattr(e, '__dict__') else str(e)
        raise HTTPException(status_code=400, detail=f"Error processing image: {error_msg}")

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("api:app", host="0.0.0.0", port=8000, reload=True)
```

■ Step 15: Run the REST API

> *Description: Launches the FastAPI backend service. Interactive Swagger documentation is automatically available at `http://127.0.0.1:8000/docs`.*

```
python serving/api.py
```

PHASE 6: WEB USER INTERFACE (STREAMLIT APP)

■ Step 16 & 17: Streamlit UI Setup & App Code (`ui/app.py`)

> *Description: Establishes the user interface directory, installs Streamlit and Requests packages, and implements an interactive web dashboard communicating with the FastAPI backend.*

```
mkdir ui
pip install streamlit requests

# ui/app.py content:
import streamlit as st
import requests
from PIL import Image
import io

st.set_page_config(page_title="Casting Defect Detection App", page_icon="⚙️",
layout="centered")
API_URL = "http://127.0.0.1:8000/predict"

st.title("⚙️ Casting Defect Detection System")
st.markdown("Upload a casting image to analyze whether it is **ok** or **defective** using
the deployed FastAPI deep learning model.")

uploaded_file = st.file_uploader("Choose a casting image...", type=["jpg", "jpeg", "png"])
if uploaded_file is not None:
    image = Image.open(uploaded_file)
    st.image(image, caption="Uploaded Casting Image", use_container_width=True)

    if st.button("🔍 Predict Defect", type="primary"):
        with st.spinner("Analyzing image via FastAPI backend..."):
            try:
                img_byte_arr = io.BytesIO()
                image.save(img_byte_arr, format=image.format if image.format else "JPEG")
                files = {"file": (uploaded_file.name, img_byte_arr.getvalue(),
uploaded_file.type)}

                response = requests.post(API_URL, files=files)
                if response.status_code == 200:
                    data = response.json()
                    score = data.get("prediction")
                    result = data.get("result")

                    st.success("Analysis Complete!")
                    col1, col2 = st.columns(2)
                    with col1:
                        st.metric(label="Prediction Probability", value=f"{score:.4f}")
                    with col2:
                        if result == "ok":
                            st.metric(label="Status", value="✅ OK (Defect-Free)")
                        else:
                            st.metric(label="Status", value="❌ Defective")
                else:
                    st.error(f"API Error ({response.status_code}): {response.text}")
            except Exception as e:
                st.error(f"Connection error: {str(e)}")
```

■ Step 18: Run the Streamlit Web Application

> *Description: Starts the Streamlit web server on port 8501, providing an intuitive browser-based interface for quality inspection operators.*

```
streamlit run ui/app.py
```

PHASE 7: MLOPS DATA VALIDATION & FEATURE STORE PIPELINE

■ Step 19, 21 & 22: Data Validation Layer (`validation/validate_data.py`)

> *Description: Installs Pandera, creates the validation module directory, and implements strict preprocessing checks to reject corrupted, oversized, or unformatted input images prior to ingestion.*

```
pip install pandera
mkdir validation

# validation/validate_data.py content:
import os
import io
from PIL import Image, UnidentifiedImageError

class DataValidationLayer:
    def __init__(self, target_size=(512, 512), max_file_size_mb=10, min_resolution=64):
        self.target_size = target_size
        self.max_file_size_mb = max_file_size_mb
        self.min_resolution = min_resolution
        self.allowed_formats = {"JPEG", "JPG", "PNG"}

    def validate_image_bytes(self, image_bytes: bytes) -> tuple[bool, str]:
        if not image_bytes:
            return False, "Received empty image payload."
        if (len(image_bytes) / (1024 * 1024)) > self.max_file_size_mb:
            return False, "Image exceeds maximum allowed size limit."
        try:
            image = Image.open(io.BytesIO(image_bytes))
            image.verify()
        except Exception as e:
            return False, f"Corrupted or invalid image format: {str(e)}"
        return True, "Validation passed successfully."

validator = DataValidationLayer()
```

■ Step 23, 24 & 25: Feature Engineering Pipeline (`features/build_features.py`)

> *Description: Installs scikit-learn, decouples feature transformation logic from training scripts into a dedicated module, and persists fitted transformers to prevent train/serving skew.*

```
pip install scikit-learn

# features/build_features.py content:
import os
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
import joblib

CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
BASE_DIR = os.path.dirname(CURRENT_DIR)
FEATURE_STORE_DIR = os.path.join(BASE_DIR, "feature_store")
os.makedirs(FEATURE_STORE_DIR, exist_ok=True)

def build_feature_transformer():
    return ColumnTransformer(
        transformers=[
            ('num', Pipeline([('scaler', StandardScaler())]), []),
            ('cat', Pipeline([('encoder', OneHotEncoder(handle_unknown='ignore',
sparse_output=False))]), [])
        ],
        remainder='drop'
    )

def process_and_persist_features(df: pd.DataFrame, is_training: bool = True):
    transformer_path = os.path.join(FEATURE_STORE_DIR, "feature_transformer.pkl")
    if is_training:
        transformer = build_feature_transformer()
        transformed = transformer.fit_transform(df) if not df.empty else np.array([])
        joblib.dump(transformer, transformer_path)
    else:
        transformer = joblib.load(transformer_path)
        transformed = transformer.transform(df) if not df.empty else np.array([])
    return transformed
```

■ Step 26 & 27: Training from Feature Store (`training/train_model_from_store.py`)

> *Description: Connects the model training pipeline directly to the MLOps feature store contract and executes neural network training.*

```
python training/train_model_from_store.py
```

PHASE 8: TRANSFER LEARNING & MODEL BENCHMARKING

■ Step 28 & 29: Transfer Learning Model (`training/train_model_transfer_model.py`)

> *Description: Implements transfer learning utilizing the pre-trained MobileNetV2 architecture with frozen base weights and a customized classification head optimized for industrial casting defect detection.*

```
import os
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.callbacks import EarlyStopping

CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
BASE_DIR = os.path.dirname(CURRENT_DIR)
DATA_DIR = os.path.join(BASE_DIR, "data", "casting_data", "casting_data")
MODEL_STORE_DIR = os.path.join(BASE_DIR, "model_store")

IMG_SIZE = (224, 224)
BATCH_SIZE = 16
EPOCHS = 5
LEARNING_RATE = 1e-4

def run_training_pipeline():
    os.makedirs(MODEL_STORE_DIR, exist_ok=True)
    train_ds = tf.keras.utils.image_dataset_from_directory(os.path.join(DATA_DIR, "train"),
image_size=IMG_SIZE, batch_size=BATCH_SIZE, shuffle=True)
    test_ds = tf.keras.utils.image_dataset_from_directory(os.path.join(DATA_DIR, "test"),
image_size=IMG_SIZE, batch_size=BATCH_SIZE, shuffle=False)

    base_model = MobileNetV2(input_shape=(224, 224, 3), include_top=False,
weights='imagenet')
    base_model.trainable = False

    model = models.Sequential([
        layers.Input(shape=(224, 224, 3)),
        layers.RandomFlip("horizontal_and_vertical"),
        layers.RandomRotation(0.2),
        layers.Rescaling(1./127.5, offset=-1),
        base_model,
        layers.GlobalAveragePooling2D(),
        layers.Dense(128, activation='relu'),
        layers.Dropout(0.4),
        layers.Dense(1, activation='sigmoid')
    ])

    model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=LEARNING_RATE),
loss='binary_crossentropy', metrics=['accuracy'])
    early_stopping = EarlyStopping(monitor='val_loss', patience=5,
restore_best_weights=True)

    model.fit(train_ds, validation_data=test_ds, epochs=EPOCHS, callbacks=[early_stopping])
    model.save(os.path.join(MODEL_STORE_DIR, "casting_defect_transfer_model.keras"))

if __name__ == "__main__":
    run_training_pipeline()
```

■ Step 30: Model Evaluation & Comparison (`compare_models.py`)

> *Description: Evaluates both the Custom CNN and MobileNetV2 Transfer Learning models against the test dataset to benchmark accuracy and loss metrics.*

```
import os
import tensorflow as tf

CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
TEST_DIR = os.path.join(CURRENT_DIR, "data", "casting_data", "casting_data", "test")
MODEL_STORE_DIR = os.path.join(CURRENT_DIR, "model_store")

def evaluate_models():
    test_ds = tf.keras.utils.image_dataset_from_directory(TEST_DIR, image_size=(224, 224),
batch_size=16, shuffle=False)

    models_to_test = {
        "Custom CNN": os.path.join(MODEL_STORE_DIR, "casting_defect_classifier.keras"),
        "Transfer Learning (MobileNetV2)": os.path.join(MODEL_STORE_DIR,
"casting_defect_transfer_model.keras")
    }

    print("Model Evaluation Results:")
    for name, path in models_to_test.items():
        if os.path.exists(path):
            model = tf.keras.models.load_model(path)
            loss, accuracy = model.evaluate(test_ds, verbose=0)
            print(f"{name}: Loss={loss:.4f}, Accuracy={accuracy*100:.2f}%")

if __name__ == "__main__":
    evaluate_models()
```

📊 Comparative Model Performance Summary

Benchmark results comparing the baseline Custom CNN versus the MobileNetV2 Transfer Learning model on the test dataset:

Model Name	Test Loss	Test Accuracy	Artifact Size
Custom CNN (Scratch)	0.6968	43.50%	44.73 MB
Transfer Learning (MobileNetV2)	**0.0804**	**98.25%**	**11.61 MB**